

TP3 - Modifications mémoire

COMLAN Theodorik

Sauvegarde

La première partie consistait à obtenir un score élevé dans le menu View highscores, sans réellement jouer une longue partie, puis de comprendre comment Bastet sauvegarde les scores.

Avec la commande ***strace -f -o trace.txt bastet*** strace on peut observer les appels système :

On voit les fichiers ouverts par le jeu :

```
22 42489 openat(AT_FDCWD, "/var/games/bastet.scores2", O_RDWR) = -1 EACCES (Permission denied)
23 42489 openat(AT_FDCWD, "/home/tomubt/.bastetscores", O_RDWR) = 4
24 42489 openat(AT_FDCWD, "/home/tomubt/.bastetscores", O_RDONLY) = 4
```

Bastet tente d'abord d'utiliser le fichier **/var/games/bastet.scores2**. Comme mon utilisateur n'a pas les droits en écriture dessus, le jeu utilise le fichier local **/home/tomubt/.bastetscores**.

Après avoir fait une courte partie avec un score inférieur à 300, on a fermé le jeu afin qu'il écrive le fichier de sauvegarde. Ensuite, on a modifié le fichier **~/.bastetscores** pour remplacer un score par une valeur élevée.

Le GIF "**demo_best_score.gif**" montre le score élevé obtenu après modification du fichier contenant le score.

Score en jeu

Pour modifier le score pendant que bastet était en cours d'exécution, on a écrit un programme C qui ouvre la mémoire du processus cible avec **/proc/\$PID/mem**.

La difficulté principale est que l'adresse du score change à chaque lancement à cause de l'ASLR et de l'allocation dynamique. Il faut donc trouver l'adresse à l'exécution. Une fois Bastet lancé dans un terminal, on peut récupérer son PID avec **pgrep -n bastet**. Le PID est ensuite passé au programme.

Le programme final est **write_in_game.c**. Il utilise deux fichiers du système /proc : **/proc/\$PID/maps** pour connaître les plages d'adresses virtuelles du processus. **/proc/\$PID/mem** pour lire et écrire directement dans la mémoire du processus.

Le programme ouvre d'abord la mémoire :

```
sprintf(path, "/proc/%d/mem", pid);  
int mem = open(path, O_RDWR);
```

Comme cet accès est protégé, il faut généralement lancer le programme avec **sudo**.

Le programme demande le score actuellement affiché dans le jeu.
Il refuse le **score 0**, car cette valeur est trop commune en mémoire et donnerait trop de faux positifs.

Ensuite, il parcourt les mappings mémoire accessibles en lecture/écriture :

```
if (perms[0] == 'r' && perms[1] == 'w')
```

Pour chaque zone mémoire, il lit des entiers avec **pread**. Si la valeur lue correspond au score affiché, l'adresse est ajoutée à la liste des candidats. Un seul scan ne suffit pas toujours, car plusieurs adresses peuvent contenir la même valeur. Pour résoudre ce problème, le programme fonctionne comme Cheat Engine :

1. On saisit le score actuel.
2. Le programme trouve toutes les adresses contenant cette valeur.
3. On gagne encore quelques points dans Bastet.
4. On saisit le nouveau score.
5. Le programme filtre les candidats en conservant uniquement les adresses qui ont évolué vers cette nouvelle valeur. La boucle continue jusqu'à ce qu'il ne reste qu'une seule adresse

Quand l'adresse unique est trouvée, le programme écrit un nouveau score avec **pwrite**.

La démonstration de cette partie est dans “**modif_score_in_game.gif**”

Truquer la génération de bloc

Dans cette partie on doit truquer la génération des blocs pour que Bastet donne toujours le même bloc.

On va utiliser **ghidra** pour modifier le binaire. Etant donné que le binaire Bastet est **strippé**, les fonctions apparaissent avec des noms génériques. Bastet étant open-source, on s'est aidé du code source pour identifier les fonctions de génération des blocs.

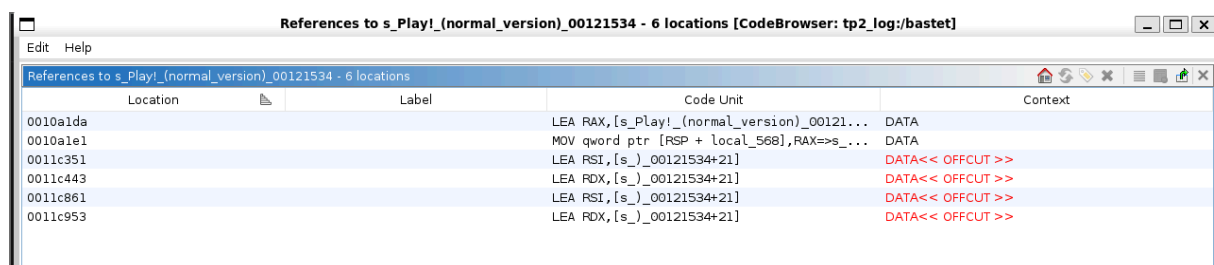
Les types de blocs sont définis dans **Block.hpp** avec un **enum BlockType**. La génération normale est dans **BastetBlockChooser.cpp**. La fonction la plus importante est **NoPreviewBlockChooser::GetNext** qui calcule le bloc le plus défavorable pour le joueur.

Pour trouver **GetNext()**, on est parti de la chaîne **"Play! (normal version)"** qui se trouve dans le main.cpp.

```
int main(int argc, char **argv){
    Ui ui;
    while(1){

        int choice=ui.MenuDialog(list_of("Play! (normal version)");
        switch(choice){
        case 0:{
            //ui.ChooseLevel();
            BastetBlockChooser bc;
            ui.Play(&bc);
            ui.HandleHighScores(difficulty_normal);
            ui.ShowHighScores(difficulty_normal);
        }
    }
}
```

En faisant **Show References** sur cette chaîne, on obtient :



The screenshot shows a CodeBrowser window titled "References to s_Play!(normal_version)_00121534 - 6 locations [CodeBrowser: tp2_log:/bastet]". The window contains a table with 5 columns: Location, Label, Code Unit, and Context. The table lists 6 references to the memory address 00121534.

Location	Label	Code Unit	Context
0010a1da		LEA RAX,[s_Play!(normal_version)_00121...	DATA
0010a1e1		MOV qword ptr [RSP + local_568],RAX=>s_...	DATA
0011c351		LEA RSI,[s_]_00121534+21]	DATA<< OFFCUT >>
0011c443		LEA RDX,[s_]_00121534+21]	DATA<< OFFCUT >>
0011c861		LEA RSI,[s_]_00121534+21]	DATA<< OFFCUT >>
0011c953		LEA RDX,[s_]_00121534+21]	DATA<< OFFCUT >>

La **première référence** est celle du menu principal que la main affiche. C'est donc elle qui nous intéresse.

En allant à cette référence, on retrouve le switch sur le choix du menu:

```
304 switch(uVar16 & 0xffffffff) {
305 case 0:
306     local_548 = &PTR_FUN_00128a88;
307     FUN_00113680(local_3f8,&local_548);
308     FUN_001141a0(local_3f8,0);
309     FUN_001130d0(local_3f8,0);
310     goto switchD_0010a691_default;
```

En comparant avec le code source, on peut faire une correspondance entre **PTR_FUN_00128a88** et **BastetBlockChooser**.

En double cliquant sur **PTR_FUN_00128a88**, on obtient

	PTR_FUN_00128a88		XREF[2]:	FUN_00109e90:0010aaf2(*), FUN_00109e90:0010aaf9(*)
00128a88	f0 bd 11 00 00 00 00 00	addr	FUN_0011bdf0	
00128a90	b0 69 11 00 00 00 00 00	addr	FUN_001169b0	
00128a98	e0 a0 11 00 00 00 00 00	addr	FUN_0011a0e0	
00128aa0	b0 9a 11 00 00 00 00 00	addr	FUN_00119ab0	

La dernière référence est la fonction qui truque la génération du prochain bloc.

Il nous suffit donc de modifier la valeur de retour (**registre EAX**) de la fonction, pour qu'elle renvoie tout le temps le même bloc.

	undefined8	Stack[-0x124... local_124 FUN_00119ab0	
00119ab0	f3 0f 1e fa	ENDBR64	
00119ab4	b8 00 00 00 00	MOV	EAX,0x0
00119ab9	c3	RET	

Ici on force l'apparition de bloc 0 (carré).

Pour finir, le GIF "**same_block_gen.gif**" contient la preuve du binaire "**bastet_pat**" avec la generation de bloc truquée.